

Inteligencia Artificial

Práctica 2: Agente multimodal de triaje de incidentes de seguridad

Oussama Marzak Faroit

Curso 2025-2026

1. Introducción: dominio elegido y motivación

El dominio elegido es el triaje de incidentes de seguridad. Cuando se produce una alerta, un analista recibe evidencias muy distintas: capturas de pantalla de una consola o un SIEM, registros de eventos en CSV o log y, a veces, informes de vulnerabilidades en PDF. El triaje consiste en entender rápidamente qué ha pasado, qué gravedad tiene y qué hacer a continuación.

La motivación es doble. Por un lado, es un dominio directamente relacionado con la Ingeniería de Ciberseguridad. Por otro, encaja de forma natural con un agente multimodal, porque las evidencias llegan en formatos diferentes y un único asistente capaz de procesarlos todos aporta valor real.

El sistema cubre tres escenarios de uso: diagnóstico de una captura de pantalla de una alerta, análisis de un registro de eventos con extracción de indicadores de compromiso, y resumen de un informe de vulnerabilidades en PDF. En los tres, el analista puede hacer preguntas de seguimiento sobre la misma evidencia.

2. Arquitectura del sistema

El sistema sigue el pipeline recomendado en el enunciado, adaptado al dominio. El flujo es el siguiente:

1. Entrada: el usuario aporta una evidencia (imagen, PDF, CSV o log) y una consulta.
2. Preprocesamiento: según el tipo de archivo se extrae su contenido con Pillow, PyMuPDF o pandas.
3. Agente de LangChain: un modelo con tool calling decide qué herramienta usar según la evidencia cargada y la consulta.
4. Herramientas: una por funcionalidad (diagnosticar la captura, analizar el registro o resumir el informe), cada una con su prompt.
5. Salida estructurada: el agente devuelve un informe de triaje con apartados fijos.
6. Interacción: el historial se conserva, lo que permite preguntas de seguimiento.

Las herramientas leen la evidencia desde el propio agente, de modo que el modelo solo decide la acción y no transporta el contenido del documento.

3. Decisiones técnicas

3.1. Herramienta de despliegue: Ollama frente a LM Studio

Se ha elegido Ollama. Funciona desde la terminal y arranca un servidor sin interfaz gráfica, expone una API REST compatible con la de OpenAI (lo que permite integrarlo con LangChain mediante el paquete oficial) y es la herramienta que se trabajó en clase. LM Studio es una alternativa válida, pero su punto fuerte es la interfaz gráfica, que aquí no aporta porque la orquestación es programática.

3.2. Modelos

Se usan dos modelos locales libres y ligeros. Como orquestador se emplea qwen2.5 de 3.000 millones de parámetros, porque admite llamadas a herramientas de forma nativa, lo que hace fiable el patrón de agente con herramientas. Para las imágenes se usa llava de 7.000 millones, que interpreta el contenido visual de las alertas. Separar los dos modelos evita el problema de que muy pocos modelos combinan bien visión y tool calling en local, así que cada uno se especializa en su tarea. Entre los dos ocupan unos 6,5 GB y caben de sobra en la GPU utilizada.

3.3. Librerías

El proyecto se apoya en varias librerías. LangChain y langchain-ollama se encargan de la orquestación y de la conexión con el modelo. PyMuPDF extrae el texto de los PDF, Pillow maneja las imágenes y pandas lee los CSV. Por último, Streamlit construye la interfaz y requests comprueba el estado del servidor de Ollama.

4. Prompt engineering

El enunciado pide prompts específicos por funcionalidad, no un prompt genérico. Se han diseñado cuatro. El primero es el prompt de sistema del agente, que fija su papel y la regla de usar la herramienta adecuada. Los otros tres son específicos de cada herramienta, uno para la imagen, otro para el registro y otro para el PDF.

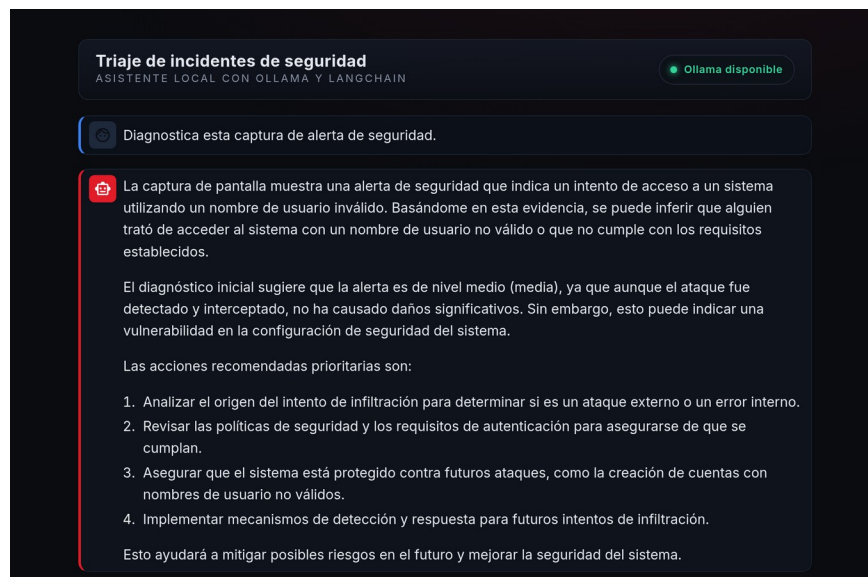
La decisión común a todos es forzar una salida estructurada (qué se observa, severidad y acciones recomendadas) y prohibir la invención de datos que no aparezcan en la evidencia. En un contexto de seguridad, un dato inventado, como una IP o un CVE que no existe, es peor que una respuesta incompleta.

5. Ejemplos de uso

Se incluyen datos de ejemplo sintéticos para los tres escenarios. A continuación se muestra cada uno con su captura real de la interfaz.

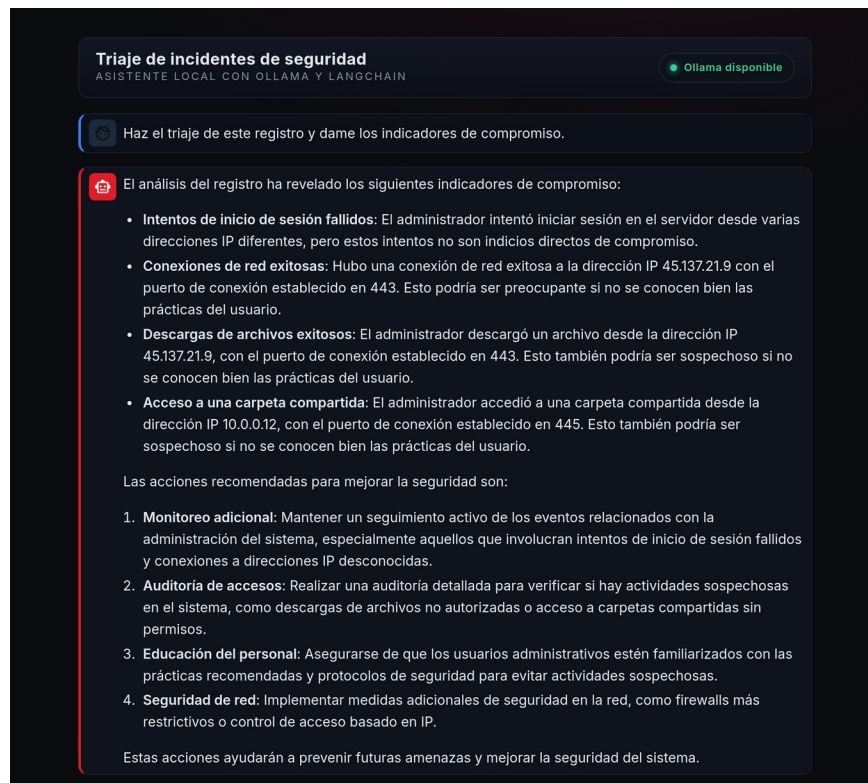
5.1. Escenario 1: diagnóstico de una captura

Entrada: una captura de una alerta de SIEM (intentos de inicio de sesión fallidos seguidos de uno correcto). El agente la envía al modelo de visión, que la interpreta como una alerta de acceso no autorizado, le asigna severidad alta y propone acciones.



5.2. Escenario 2: análisis de un registro de eventos

Entrada: un CSV con accesos por RDP y una conexión saliente posterior. El agente extrae como indicadores de compromiso las direcciones IP sospechosas, describe los eventos y recomienda bloquear la IP de origen de los intentos fallidos.



5.3. Escenario 3: resumen de un informe de vulnerabilidades

Entrada: un informe en PDF con varias vulnerabilidades. El agente las lista con su CVE y criticidad (entre ellas Log4Shell, con CVSS 10) y propone un plan de remediación priorizado. En la pregunta de seguimiento mantiene el contexto.



6. Limitaciones y mejoras futuras

En cuanto a las limitaciones, los modelos locales pequeños son razonables para una primera orientación, pero menos precisos que los de pago, y a veces se equivocan con imágenes de texto pequeño o con logs muy largos. La ventana de contexto obliga a recortar los documentos más extensos. Además, el agente razona sobre una sola evidencia a la vez. En cualquier caso, el triaje es orientativo y no sustituye a un especialista ni constituye una conclusión forense.

De cara al futuro, una primera mejora sería incorporar RAG sobre logs largos o sobre una base de datos de vulnerabilidades, para fundamentar mejor las respuestas. También resultaría útil permitir la correlación de varias evidencias del mismo incidente y añadir OCR como apoyo al modelo de visión. Por último, convendría exportar el informe de triaje a un formato estándar para integrarlo con el flujo de respuesta a incidentes.